

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente División
de Ciencias de la Ingeniería
Ing. Moisés Granados Guevara
Introducción a la programación IPC2



Manual de Técnico

Sistema de Gestión de

Restaurante

Guillermo Emanuel Montejo Martínez
Carné: 202430624

Quetzaltenango, 13 de agosto de 2026

Descripción General y Arquitecturas

"JavaBeans Café" es una aplicación de escritorio orientada a transacciones y desarrollada en lenguaje Java. Fue diseñada como una herramienta integral para la gestión administrativa de cafeterías. Sus módulos principales permiten automatizar el control de inventarios, costear productos mediante recetas, administrar el recurso humano (incluyendo la generación de nóminas) y procesar el cobro de cuentas por mesa.

Tecnología y herramientas base:

Java Development Kit (JDK): Entorno esencial que proporciona las herramientas necesarias para compilar el código fuente y ejecutar la aplicación a través de la Máquina Virtual de Java (JVM).

- **Java Swing:** Framework gráfico empleado para el diseño de la Interfaz Gráfica de Usuario (GUI). Se implementaron contenedores de alto nivel como JDesktopPane y JInternalFrame para lograr una experiencia de usuario dinámica mediante un entorno de múltiples ventanas.
- **MySQL Server:** Sistema de Gestión de Bases de Datos Relacionales (RDBMS) seleccionado para garantizar la persistencia, concurrencia e integridad referencial de toda la información del negocio.
- **JDBC (Java Database Connectivity):** API nativa implementada para establecer la comunicación directa entre la aplicación y el motor MySQL, permitiendo la ejecución de consultas SQL crudas y el manejo estructurado de transacciones.
- **Apache Maven:** Herramienta de automatización y gestión de proyectos utilizada para administrar las dependencias externas (como el controlador de MySQL) y compilar el proyecto en un artefacto ejecutable .jar.

Arquitectura

El sistema fue diseñado respetando el paradigma de la Programación Orientada a Objetos (POO), garantizando una alta cohesión y un bajo acoplamiento. A diferencia de arquitecturas básicas, el proyecto se estructuró mediante un Patrón DAO (Data Access Object) y el uso de genéricos, dividiendo las responsabilidades en los siguientes paquetes lógicos:

1. **Paquete entidades (Modelos de Dominio):** Clases puras de Java (ej. Empleado.java, Insumo.java, Cuenta.java) que representan la abstracción de las tablas de la base de datos. Encapsulan la información mediante atributos privados, constructores y métodos *Getters* y *Setters*, apoyándose en enumeraciones (Enums) para garantizar la integridad de datos predefinidos (ej. EstadoMesa, RolEmpleado).
2. **Paquete acceso :** Capa encargada exclusivamente de la persistencia y comunicación con MySQL. Centraliza las sentencias SQL mediante la interfaz genérica `MantenimientoAcceso<T>`, la cual obliga a estandarizar las operaciones CRUD (insertar, actualizar, listarTodos) en clases especializadas como `CuentaAcceso.java` e `InsumoAcceso.java`.
3. **Paquete estructuras (TDA Propios):** Implementación de Tipos de Datos Abstractos personalizados desde cero, como `ListaDinamica<T>` y `NodoGenerico<T>`, evitando la dependencia total de `java.util`. Esto optimiza el manejo de la memoria en tiempo de ejecución.

4. **Paquetes utilidades y reportes:** Módulos de soporte técnico. Incluyen clases renderizadoras visuales para las tablas Swing (ej. alertas de stock visuales) y un motor de generación de reportes transaccionales con salida directa a HTML.
5. **Capa de Vistas (Swing):** Componentes visuales gráficos diseñados para la captura de eventos del usuario, enviando los requerimientos a la capa de acceso.

Gestion Transaccional

Para garantizar la atomicidad y consistencia de la base de datos (Propiedades ACID), el sistema abandona el AutoCommit predeterminado de JDBC en operaciones complejas. Métodos como `levantarCuentaTransaccional` y `pagarCuentaTransaccional` agrupan sentencias secuenciales (inserción de cuenta, detalle de cuenta, actualización de estado de mesa y rebaja de inventario vía receta). Si alguna sentencia falla, el sistema ejecuta un rollback, asegurando que no existan datos huérfanos.

Patrones de diseño aplicados

Para resolver problemas recurrentes de arquitectura y asegurar la escalabilidad del sistema, "JavaBeans Café" implementa patrones de diseño de software reconocidos en la industria:

- **Patrón Singleton (Instancia Única):** Aplicado en la clase genérica de configuración `Conexion`. Este patrón garantiza que el sistema genere y consuma una única instancia del manejador de base de datos durante toda la ejecución. Esto optimiza drásticamente el consumo de memoria y previene el colapso del servidor de MySQL por exceso de peticiones simultáneas.
- **Patrón DAO (Data Access Object):** Estructura fundamental del paquete acceso. Se separó por completo la lógica de negocio y presentación de la lógica de acceso a datos. A través de la interfaz genérica `MantenimientoAcceso<T>`, se establecieron contratos estrictos de programación para las operaciones CRUD, implementados de forma específica en clases como `CuentaAcceso.java`, `InsumoAcceso.java` y `EmpleadoAcceso.java`.

Algoritmos y estructuras de datos personalizadas

El sistema fue desarrollado con un enfoque profundo en el manejo eficiente de la memoria en tiempo de ejecución. En lugar de depender exclusivamente de las colecciones estándar de Java (como `ArrayList`), se implementaron Tipos de Datos Abstractos (TDA) desde cero en el paquete estructuras:

- **Implementación de Nodos y Listas Dinámicas:** Se construyó la clase `NodoGenerico<T>` y `ListaDinamica<T>`. Estas estructuras enlazadas permiten almacenar datos en la memoria Heap de forma no contigua, facilitando el crecimiento dinámico de los elementos sin el costo computacional de redimensionar arreglos estáticos.
- **Aplicación Práctica:** Esta estructura genérica es el núcleo temporal del sistema de ventas. Se utiliza intensivamente para manejar los carritos de compras y múltiples objetos iterables (ej. `ListaDinamica<Object[]>` `detallesCarrito`), pasando información limpia y tipada desde las vistas de Swing hacia las capas de acceso a bases de datos.

Seguridad y manejo de errores

El sistema cuenta con mecanismos de defensa técnica para garantizar la resiliencia y la consistencia de los datos ante fallos inesperados o intentos de manipulación:

- **Prevención de Inyección SQL (SQLi):** Todas las consultas a la base de datos utilizan la interfaz `PreparedStatement` de JDBC. Al utilizar parámetros precompilados (?) en lugar de concatenación directa de cadenas, el sistema "escapa" cualquier entrada maliciosa, blindando la base de datos contra ataques de inyección.
- **Propiedades ACID y Control Transaccional Manual:** En operaciones críticas (como el cobro de cuentas o el registro de compras de inventario), se deshabilitó el autoguardado de JDBC (`conn.setAutoCommit(false);`). Clases como `CuentaAcceso` agrupan operaciones secuenciales (crear la cuenta, guardar detalles, actualizar la mesa y descontar la receta del inventario). Si ocurre un error lógico o de conexión en cualquier punto, el bloque `catch` ejecuta un `conn.rollback()`, revirtiendo todos los cambios y garantizando que no existan datos huérfanos.
- **Gestión Segura de Recursos (Memory Leaks):** Se estandarizó el uso de bloques `try-with-resources` (`try (Connection conn = Conexion.getConnection(); PreparedStatement stmt = ...)`). Este enfoque de programación avanzada garantiza el cierre automático e inmediato de conexiones, *Statements* y *ResultSets* al finalizar una operación, liberando memoria y previniendo bloqueos en el motor de base de datos.

Guía de despliegue e instalación

Para ejecutar el sistema "JavaBeans Café" en un nuevo entorno de producción, se deben cumplir los siguientes requisitos y pasos de configuración:

1. Requisitos del Sistema:

- **Entorno de Ejecución:** Java Runtime Environment (JRE) / JDK versión 17 o superior.
- **Base de Datos:** MySQL Server versión 8.0+.
- **Librerías externas:** Archivo `mysql-connector-j.jar` configurado en el *Build Path* del proyecto (o gestionado automáticamente vía `pom.xml` si se usa Maven).

2. Restauración del Esquema Relacional:

- Ejecutar el script SQL proporcionado (`Diccionario_Datos_Fisico.sql`) en la instancia local de MySQL (a través de MySQL Workbench o consola).
- Este script creará la base de datos `javabeans_cafe`, junto con sus tablas normalizadas, enumeraciones (ENUM), llaves primarias (PK) y llaves foráneas (FK).

3. Configuración de Credenciales de Conexión:

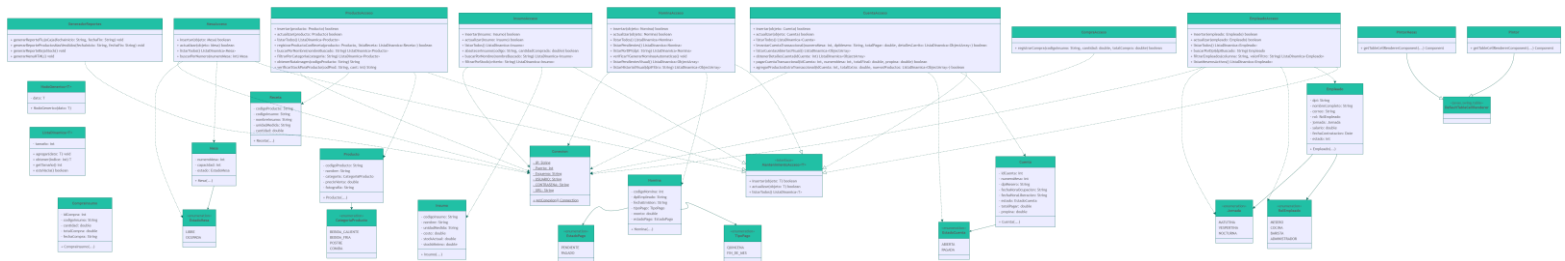
- Abrir el código fuente y ubicar el paquete conexion.
- Dentro del archivo Conexion.java, actualizar los parámetros estáticos con las credenciales locales del servidor destino:
 - IP: Dirección del servidor (ej. localhost o 127.0.0.1).
 - Puerto: Puerto de escucha de MySQL (usualmente 3306).
 - USUARIO: Usuario con privilegios de lectura/escritura.
 - CONTRASEÑA: Contraseña del usuario correspondiente.

4. Ejecución:

- Compilar el proyecto ejecutando la clase principal (*Main*) que inicializa la interfaz Swing, o construir el artefacto .jar ejecutable para el cliente final.

Diagrama de clases (UML)

El diseño estructural del sistema se representa en el siguiente diagrama, mostrando las dependencias, implementaciones de interfaces y cardinalidades entre las entidades lógicas.



Modelo Relacional y Mapeo Físico de Base de Datos

El sistema depende de una base de datos relacional altamente normalizada para evitar redundancias y garantizar la integridad referencial. A continuación se detallan los esquemas conceptuales y físicos:

Diagrama Entidad Relación

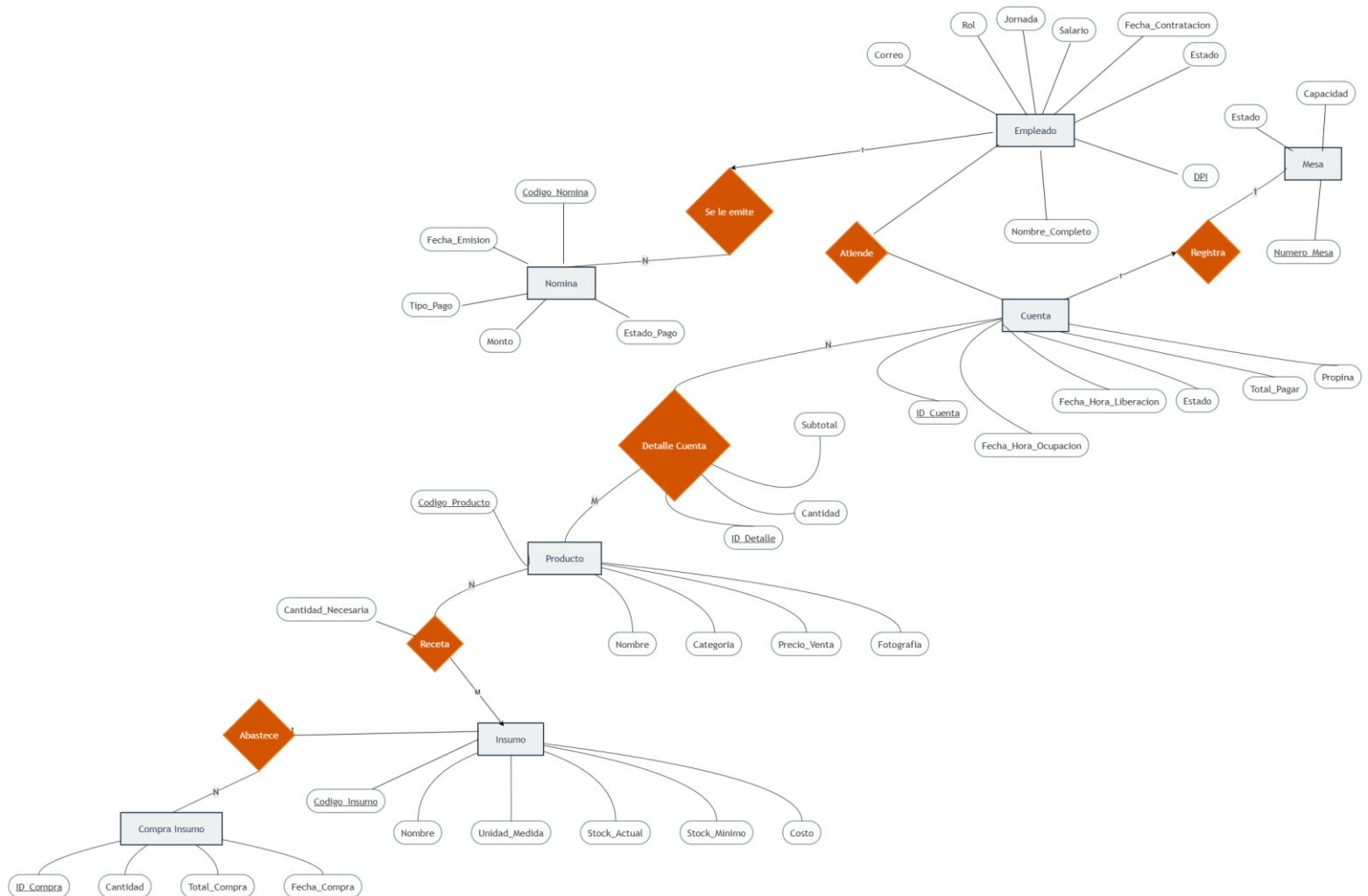
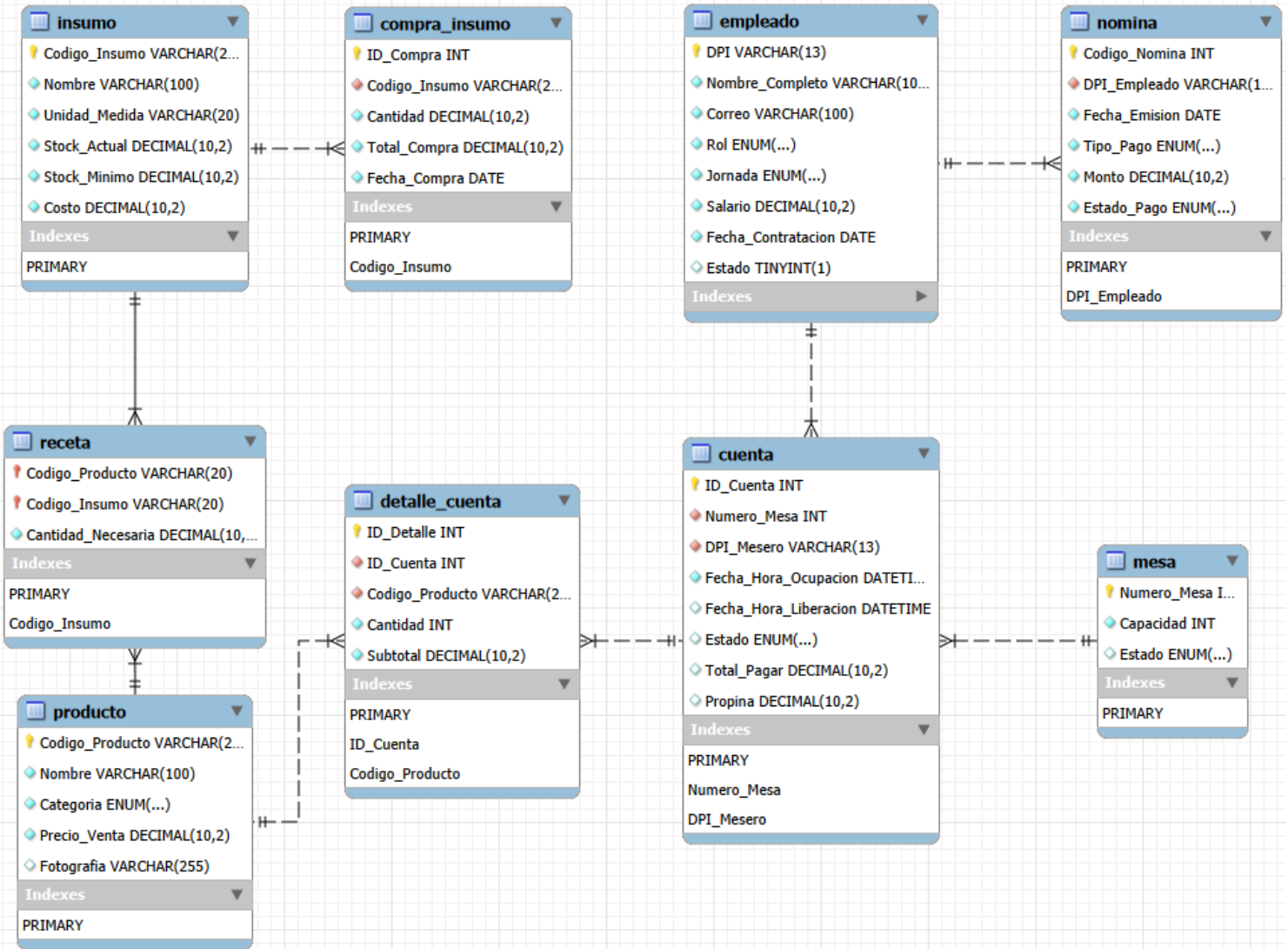


Diagrama de tablas



Mapecto Físico de la Data Base

Table Name: empleado		Schema: javabeanscafe									
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression	
DPI	VARCHAR(13)	☒	☒	☐	☐	☐	☐	☐	☐		
Nombre_Completo	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐		
Correo	VARCHAR(100)	☐	☒	☒	☐	☐	☐	☐	☐		
Rol	ENUM('MESERO', 'COCINA', 'BARISTA', 'ADMINISTRADOR')	☐	☒	☐	☐	☐	☐	☐	☐		
Jornada	ENUM('MATUTINA', 'VESPERTINA', 'NOCTURNA')	☐	☒	☐	☐	☐	☐	☐	☐		
Salario	DECIMAL(10,2)	☐	☒	☐	☐	☐	☐	☐	☐		
Fecha_Contratacion	DATE	☐	☒	☐	☐	☐	☐	☐	☐		
Estado	TINYINT(1)	☐	☐	☐	☐	☐	☐	☐	☐	'1'	

Table Name: nomina		Schema: javabeanscafe									
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression	
Codigo_Nomina	INT	☒	☒	☐	☐	☐	☐	☒	☐		
DPI_Empleado	VARCHAR(13)	☐	☒	☐	☐	☐	☐	☐	☐		
Fecha_Emision	DATE	☐	☒	☐	☐	☐	☐	☐	☐		
Tipo_Pago	ENUM('QUINCENA', 'FIN_DE_MES')	☐	☒	☐	☐	☐	☐	☐	☐		
Monto	DECIMAL(10,2)	☐	☒	☐	☐	☐	☐	☐	☐		
Estado_Pago	ENUM('PENDIENTE', 'PAGADO')	☐	☒	☐	☐	☐	☐	☐	☐		

Table Name: cuenta		Schema: javabeanscafe									
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression	
ID_Cuenta	INT	☒	☒	☐	☐	☐	☐	☒	☐		
Numero_Mesa	INT	☐	☒	☐	☐	☐	☐	☐	☐		
DPI_Mesero	VARCHAR(13)	☐	☒	☐	☐	☐	☐	☐	☐		
Fecha_Hora_Ocupacion	DATETIME	☐	☒	☐	☐	☐	☐	☐	☐		
Fecha_Hora_Liberacion	DATETIME	☐	☒	☐	☐	☐	☐	☐	☐		
Estado	ENUM('ABIERTA', 'PAGADA')	☐	☐	☐	☐	☐	☐	☐	☐	'ABIERTA'	
Total_Pagar	DECIMAL(10,2)	☐	☐	☐	☐	☐	☐	☐	☐	'0.00'	
Propina	DECIMAL(10,2)	☐	☐	☐	☐	☐	☐	☐	☐	'0.00'	



Table Name: compra_insumo

[illegible]